– $\mathsf{Dec}(k, \boldsymbol{h}, st) \to m/\perp$: A deterministic decoding algorithm that takes a key k, a history $\boldsymbol{h}$, and a stegotext st, and outputs a message m or a rejection symbol $\perp$.

Correctness. A steganographic scheme is correct, if decoding an honestly generated stegotext yields the hidden message with overwhelming probability.

Definition 2. *A steganographic scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be correct if for $\lambda \in \mathbb{N}$, any message m, and any history $\boldsymbol{h} \in \Sigma^*$ it holds that the following function is negligible in λ:*

$$\Pr\left[m' \neq m \;:\; \begin{array}{c} k \leftarrow\$ \mathsf{KeyGen}(1^\lambda) \\ st \leftarrow\$ \mathsf{Enc}(k, m, \boldsymbol{h}) \\ m' \leftarrow \mathsf{Dec}(k, \boldsymbol{h}, st) \end{array}\right].$$

Security. Informally, a steganographic scheme is said to be secure if no efficient adversary can detect whether communication over the channel Ch bears a hidden message or not. This is formalized by granting the adversary oracle access to either an encoding oracle $\mathsf{Enc}(k, \cdot, \cdot)$ or to a channel-sampling oracle $\mathcal{O}_{\mathcal{D}}(\cdot, \cdot)$. The oracle $\mathcal{O}$ takes in a message m and a history $\boldsymbol{h}$. It ignores m and replies with a response $\boldsymbol{a} \in \Sigma^*$ sampled according to the distribution $\mathcal{D}(\boldsymbol{h})$. An efficient adversary should not be able to tell whether it is given access to $\mathsf{Enc}(k, \cdot, \cdot)$ or to $\mathcal{O}_{\mathcal{D}}(\cdot, \cdot)$.

Definition 3. *A steganographic scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be* secure against chosen hidden-text attacks *if for every probabilistic polynomial-time adversary A, there exists a negligible function $\nu(\cdot)$ such that*

$$\mathsf{Adv}^{\mathrm{cha}}_{D,\Pi}(\lambda) := \left|\Pr\left[A(1^\lambda)^{\mathsf{Enc}(k,\cdot,\cdot)} = 1\right] - \Pr\left[A(1^\lambda)^{\mathcal{O}_{\mathcal{D}}(\cdot,\cdot)} = 1\right]\right| \leq \nu(\lambda),$$

for all sufficiently large $\lambda \in \mathbb{N}$, where the probabilities are taken over $k \leftarrow\$ \mathsf{KeyGen}(1^\lambda)$, the randomness of A and the randomness of the oracles.

5.2 Weak Robustness

Under the weak threat model, robustness ensures that the hidden message can be successfully recovered even if the stegotext is altered by a function $f \in \mathcal{F}_{k,\epsilon}$, where $\mathcal{F}_{k,\epsilon}$ denotes the family of locally consistent tampering functions.

Definition 4 (Weak Robustness). *A steganographic scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be $\mathcal{F}_{k,\epsilon}$-**robust** if for every probabilistic polynomial-time adversary A, there exists a negligible function $\nu(\cdot)$ such that:*

$$\mathsf{Adv}^{\mathrm{rbst}}_{A,\Pi,\mathcal{F}_{k,\epsilon}}(\lambda) := \Pr\left[\mathbf{WeakRobust}_{\mathcal{A},\Pi,\mathcal{F}_{k,\epsilon}}(\lambda) = 1\right] \leq \nu(\lambda),$$

for all sufficiently large $\lambda \in \mathbb{N}$. Figure 1 contains the definition of the weak robustness security game.

$\textbf{WeakRobust}_{\mathcal{A},\Pi,\mathcal{F}_{k,\epsilon}}(\lambda)$

1 : $(\boldsymbol{h}, m, \mathsf{state}) \leftarrow\$ A(1^\lambda)$
2 : $k \leftarrow\$ \mathsf{KeyGen}(1^\lambda)$
3 : $st \leftarrow\$ \mathsf{Enc}(k, m, \boldsymbol{h})$
4 : $(st', f) \leftarrow\$ A(\mathsf{state}, st)$
5 : **if** $f \notin \mathcal{F}_{k,\epsilon}$ **then return** 0
6 : $m' \leftarrow \mathsf{Dec}(k, \boldsymbol{h}, st')$
7 : **if** $m \neq m'$ **then return** 1 **else return** 0

Fig. 1: The weak robustness security game for a steganographic scheme Π, an adversary A, and a tampering function family $\mathcal{F}_{k,\epsilon}$.

$\textbf{StrongRobust}_{\mathcal{A},\Pi,\delta}(\lambda)$

1 : $(\boldsymbol{h}, m, \mathsf{state}) \leftarrow\$ A(1^\lambda)$
2 : $k \leftarrow\$ \mathsf{KeyGen}(1^\lambda)$
3 : $st \leftarrow\$ \mathsf{Enc}(k, m, \boldsymbol{h})$
4 : $st' \leftarrow\$ A(\mathsf{state}, st)$
5 : **if** $\text{distance}(E(st), E(st')) > \delta$ **then return** 0
6 : $m' \leftarrow \mathsf{Dec}(k, \boldsymbol{h}, st')$
7 : **if** $m \neq m'$ **then return** 1 **else return** 0

Fig. 2: The strong robustness security game for a steganographic scheme Π, an adversary A, and an embedding distance threshold δ.

5.3 Strong Robustness

Under the strong threat model, robustness ensures that the hidden message can be successfully recovered even if the stegotext is paraphrased, as long as the paraphrased text remains semantically similar to the original. This similarity is measured by the proximity of their embeddings in the embedding space.

Definition 5 (Strong Robustness). *A steganographic scheme $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ is said to be* **δ-robust** *if for every probabilistic polynomial-time adversary A, there exists a negligible function $\nu(\cdot)$ such that:*

$$\mathsf{Adv}^{\text{rbst}}_{A,\Pi,\delta}(\lambda) := \Pr\left[\textbf{StrongRobust}_{\mathcal{A},\Pi,\delta}(\lambda) = 1\right] \leq \nu(\lambda),$$

where $\delta > 0$ is a distance threshold such that distance$(E(st), E(st')) \leq \delta$ ensures the embedding of the tampered stegotext st' is close to that of the original st. Figure 2 contains the definition of the strong robustness security game.

6 Constructing Robust Steganographic Schemes

In this section, we present our schemes for robust steganography relying on LLMs. We design a scheme based on using multiple watermarks to embed information into a covertext that satisfies

the conditions needed for security under the weak robustness model and design a scheme based on rejection sampling the hash outputs of text embeddings that satisfies the conditions needed for security under the strong robustness model. We additionally prove their correctness and security for their respective threat models.

6.1 Watermarking Scheme for Weak Robustness

Our approach builds on prior work, using techniques that embed a watermark as a primitive into text generation, where the presence of a watermark can be interpreted as encoding a single bit of information. We extend this concept to encode multiple bits, leveraging statistical detection techniques (Appendix A) to determine which watermarking keys were used. This enables reliable message recovery for multi-bit messages while ensuring covertexts are indistinguishable from randomly sampled messages in the covertext space.

Encoding Process. To embed a message m, we generate text using an LLM while perturbing token probabilities based on k and m. For simplicity, assume we encode messages $m \in \{0,1\}^\ell$ of bounded length ℓ, with the secret key structured as $k = k_1 \| \cdots \| k_\ell$, where each k_i is an independent watermarking key.[2] Conceptually, for a message $m = m_1 m_2 \cdots m_\ell$, the generated text is influenced by the watermarking keys $\{k_i\}_{i:m_i=1}$ but remains independent of $\{k_i\}_{i:m_i=0}$.

More formally, during text generation, when sampling the jth token T_j, a pseudorandom function (PRF) F which outputs a bitstring in $\{0,1\}^N$ (where N is the vocab size of the LLM) is applied to the preceding c-gram of tokens:

$$r = F_k(T_{j-c}, \ldots, T_{j-1})$$

The output r determines how the probability distribution over T_j is perturbed—boosting tokens that correspond to $r_i = 1$ while suppressing tokens that correspond to $r_i = 0$. This ensures that subtle modifications to token likelihoods encode information in a way that is recoverable but not easily detectable without the key k.

Addressing Token Reuse and Detection Risks. A challenge in steganography is ensuring that the perturbations do not create detectable statistical artifacts. If c is too small, certain c-grams may repeat frequently, revealing that the text distribution has been altered. To mitigate this, we introduce a salting mechanism: both sender and receiver maintain a deterministic state s derived from the conversation history (e.g., the number of exchanged messages). Instead of computing $F_k(T_{j-c}, \ldots, T_{j-1})$ directly, we use:

$$r = F_k(s, T_{j-c}, \ldots, T_{j-1})$$

This ensures that even for the same c-gram, different conversations produce different perturbations, reducing the likelihood of detection.

Across different messages, varying the salt value means that repeated c-grams are unlikely to receive the same perturbation. Thus, an adversary attempting to detect a persistent watermark by analyzing recurring patterns will face increased difficulty, as different texts will exhibit different perturbations even under identical generation conditions.

[2] For longer messages, k can be derived using a pseudorandom function F, setting $k_i = F_k(i)$.